Exercise 9

1. SPI basics (Microchip Studio)

In this task, you will build a communication between Arduino MEGA and Arduino UNO using Serial Peripheral Interface (SPI). Arduino MEGA will act as master, and Arduino UNO as a slave.

Build the circuit using these components:
- Arduino MEGA
- Arduino UNO
- Wires

Connect the Arduino MEGA to Arduino UNO via SPI (MISO, MOSI, SCK, SS). Find the corresponding pins in both boards by checking the Pinout diagram (available in Moodle).
Connect the ground of Arduino MEGA to the ground of Arduino UNO.

In this exercise, you must build 2 files, 1 for Arduino MEGA (Master) and 1 for Arduino UNO (Slave).

Code for Arduino MEGA (Master):

1. Build communication between your computer and Arduino MEGA using USART. The step is similar to the Exercise 8.
2. In the main(void):
   - After you initialize the UART and redirect the *stdout* and *stdin*, set the pin and port SS, MOSI, and SCK as outputs:
     ```
     DDRB |= (1 << PBn) | (1 << PBn) | (1 << PBn);
     ```
   - Enable the SPI, master/slave select, and clock rate at fosc/16 (1 MHz) using SPI Control Register (SPCR). See the datasheet page 197-198.
     ```
     SPCR |= (1 << SPE) | (1 << ...) | (1 << ...);
     ```
   - Create variable of data array that will be transferred to the slave, and received from slave:
     ```
     unsigned char spi_send_data[20] = " ...write what to transfer to slave... \n\r"; //to slave
     unsigned char spi_receive_data[20]; //from slave
     ```
3. In the while loop:
   - Set the SS in master to LOW to enable the slave to transfer the data:
     ```
     PORTB &= ~(1 << PBn); // SS low, enable the slave device
     ```

- Use *for-loop* to go through the data array, sending the bytes and receiving data to an array. Use SPI Data Register (SPDR) to send and receive bytes:

```
for(int8_t spi_data_index = 0; spi_data_index < sizeof(spi_send_data); spi_data_index++)
//to read data from slave
{
        SPDR = spi_send_data[spi_data_index]; // Use SPI data register (SPDR) to send byte of
        data

        /* wait until the transmission is complete */
        while(!(SPSR & (1 << SPIF))) //see example datasheet p.193, and datasheet at p.198.
        {
                ;
        }

        spi_receive_data[spi_data_index] = SPDR; // receive byte from the SPI data register
         }
```

- After the transmission of data is finished, set the SS to HIGH to disable the slave

```
PORTB |= (1 << PBn); // SS HIGH to disable the slave device
```

- Print the received data using `printf(spi_receive_data);`

<u>Code for Arduino UNO (Slave):</u>

1. Build communication between your computer and Arduino UNO using USART. The step is similar to the Exercise 8.

2. <u>In the main(void):</u>
   - After you initialize the UART and redirect the `stdout` and `stdin`, set the pin and port of MISO as an output:

     ```
     DDRB |= (1 << PBn);
     ```

   - Enable the SPI using SPI Control Register (SPCR). See the datasheet page 167:

     ```
     SPCR |= (1 << ...);
     ```

   - Create variable of data array that will be transfer to the master, and received from master:

     ```
     char spi_send_data[20] = "...write what to transfer to master...\n\r"; //to master
     char spi_receive_data[20]; //from master
     ```

3. <u>In the while loop:</u>

   - Use *for-loop* to go through the data array, sending the bytes and receiving data to an array. Use SPI Data Register (SPDR) to send and receive bytes. You can use the same code as you have been written in the master.

   - Print the received data using `printf()`.

<u>NOTE: After you build the code, upload and print out the data using PuTTY. You need to open two PuTTY windows: one for Arduino MEGA and one for Arduino UNO.</u>

2. SPI ISR registers (Microchip Studio)

In this task, use the interrupt service routine (ISR) to execute the same functionality as in the previous task. You should use the master to poll the data to the slave. The slave then executes the interrupt in an ISR handle.

The circuit remains the same with previous task. The Arduino MEGA is used as master and Arduino UNO is used as slave.

In the slave, you receive bytes from master and send bytes in the ISR until you receive an ending character (e.g. '\r') and then print the message using USART. Note that the *ISR* routine executes after a byte transfer is complete. In the main (void) of Arduino MEGA, there is a command for sending and receiving bytes. However, the process of sending bytes to slave can be so fast, which cause trouble in the data transmission. Therefore, you can add small delay with _delay_us() after the master loads its SPDR and before it reads the received data from SPDR.

Code for the master:

The code for master also remains almost the same.

After setting the SS to low, the code is a bit different than before:

```
for(int8_t spi_data_index = 0; spi_data_index < sizeof(spi_send_data); spi_data_index++) //to read data from slave
 {
     SPDR = spi_send_data[spi_data_index]; // Use SPI data register (SPDR) to send byte of data

     _delay_us(5); // DIFFERENCE HERE
     /* wait until the transmission is complete */
     while(!(SPSR & (1 << SPIF))) //see example datasheet p.193, and datasheet at p.198.
     {
         ;
     }
     _delay_us(5); // DIFFERENCE HERE
     spi_receive_data[spi_data_index - 1 ] = SPDR; //Counting from minus 1. Receive byte from the SPI data register
  }
```

- After the transmission of data is finished, set the SS to HIGH to disable the slave

  ```
  PORTB |= (1 << PBn); // SS HIGH to disable the slave device
  ```

- Print the received data using `printf(spi_receive_data);`
- Add delay using `_delay_ms();`

Code for the slave:

1. Build communication between your computer and Arduino UNO using USART. The step is similar to the Exercise 8 until the stream function of UART.
   In this case, **remember** to `#include` interrupt and Boolean in the header using `<avr/interrupt.h>` and `<stdbool.h>`

2. Create variable of data array that will be transferred to the master, received from master, and stored using SPI communication.

```
unsigned char spi_send_data[20]= " ...write what to transfer to master... \n\r";
unsigned char spi_receive_data[20];
volatile bool transfer_complete = false;
volatile int8_t spi_index = false;
volatile int8_t spi_receive_index = false;
```

3. Access the interrupt function using ISR.

```
ISR
(SPI_STC_vect)
{
    unsigned char spi_interrupt_byte = SPDR;
    unsigned char transfer_end_check = '\r';
    SPDR = spi_send_data[spi_index]; // from zero
    spi_index++; // to ++

    if (spi_receive_index < sizeof(spi_receive_data)){//store interrupt byte in receive data
        spi_receive_data[spi_receive_index++] = spi_interrupt_byte;

        //check if the received byte is '\r'
        if (spi_interrupt_byte == transfer_end_check){
            transfer_complete = true;
        }
    }
}
```

4. In the main (void):
   - After you initialize the UART and redirect the *stdout* and *stdin*, set the pin and port of MISO as an output:

     ```
     DDRB  |= (1 << PBn);
     ```

   - Enable the SPI and SPI Interrupt Enable using SPI Control Register (SPCR). See the datasheet page 167. Also, set the SPDR to 0. See datasheet page 169.

     ```
     SPCR  |= (1 << ...)| (1 << ...);
     SPDR = 0;
     ```

   - Enable the global interrupt using `sei();`

5. In the while loop:
   Send the data to master and receive data from master.

```
if (transfer_complete == true){
printf(spi_receive_data);
transfer_complete = false;
spi_index = false;
spi_receive_index = false;
}
```

NOTE: After you build the code, upload and print out the data using PuTTY. You need to open two PuTTY windows: one for Arduino MEGA and one for Arduino UNO.